

인 수 인 계

하루마을 개발 인수인계서

일기를 쓰면 그날의 내가 캐릭터가 되어 마을에 남는
2.5D 캐주얼 생활 게임

저장소	github.com/rmsdnlgh00/diary · 로컬 ~/github/diary
스택	React 19 · TypeScript · Vite 6 · Zustand (게임 엔진 없음)
코드 규모	TS/TSX 48개 파일, 약 2,790줄
진행 상태	일기 → 감정 → 캐릭터 생성 · 자유 배회 · 대화 · 4방향 걷기까지 동작
작성일	2026년 9월 22일

1. 이 문서를 읽는 사람에게

이 프로젝트를 처음 보는 개발자(또는 AI)를 위해 썼습니다. 이미 내려진 결정과 이미 측정한 사실을 먼저 정리했으니, 같은 조사를 반복하지 말고 여기서부터 이어가면 됩니다. 특히 6장(에셋 파이프라인)과 9장(함정)은 먼저 읽어 주세요. 모르고 건드리면 되돌리기 번거로운 부분입니다.

게임 개요

사용자가 매일 일기를 쓰면 그 텍스트에서 감정을 뽑아내고, 그 날짜의 캐릭터가 마을에 하나씩 생깁니다. 9월이면 최대 30명입니다. 캐릭터들은 스스로 돌아다니다가 가까워지면 멈춰 서서 대화하고, 클릭하면 그날의 일기를 볼 수 있습니다. 같은 날짜를 다시 쓰면 캐릭터를 새로 만들지 않고 표정만 바꿉니다.



현재 실행 화면 — 주민 4명이 배회 중이고 둘이 대화하고 있습니다

2. 기술 스택과 렌더링 방식

게임 엔진을 쓰지 않습니다. Canvas도 WebGL도 없습니다. 캐릭터 한 명이 `<div>` 하나이고, 스프라이트는 CSS `background-position` 으로 칸을 옮겨 그림니다. 런타임 의존성은 `react`, `react-dom`, `zustand` 셋뿐입니다.

역할	무엇으로
UI · 렌더링	React 19 + TypeScript 5.8
빌드	Vite 6 (Node 22.9 환경이라 Vite 8 대신 6으로 고정)
상태	Zustand 5 — <code>gameStore</code> (게임), <code>debugStore</code> (개발 패널)
그리기	DOM + CSS transform / z-index
게임 루프	<code>requestAnimationFrame</code> 한 개 (<code>useVillagers.ts</code>)
저장	<code>localStorage</code> (<code>haru-village:v1</code>)
에셋 도구	<code>puppeteer-core</code> — 시스템 Chrome으로 스프라이트를 뭉습니다 (devDependency)

왜 DOM인가

일기 작성창·모달 같은 UI와 게임 화면이 섞이는 게임이라, 둘을 같은 레이어 체계 안에 두는 편이 유리했습니다. 클릭 판정이 공짜이고 (`<button>`), DevTools로 캐릭터를 직접 선택해 좌표를 읽을 수 있어 검증도 쉽습니다. 실제로 "이동 방향과 표시 중인 스프라이트가 일치 하는가"를 160회 샘플링해 확인한 것도 DOM을 읽어서 했습니다.

한계도 분명합니다. 캐릭터가 30~50명을 넘어가면 매 프레임 그만큼의 엘리먼트 스타일이 바뀌므로 부담이 옵니다. 파티클이나 블렌딩 효과도 어렵습니다. 그때는 **월드 레이어만 Canvas로 옮기고 UI는 React로 두면** 됩니다. `systems/` 가 React를 전혀 import하지 않는 순수 TypeScript라 로직은 그대로 재사용할 수 있습니다.

실행

```
npm install
npm run dev           # 개발 서버
npm run build         # 타입 검사 + 프로덕션 빌드
npm run pack-sprites  # 캐릭터 낱장 PNG -> 스프라이트 시트 + 보장값 (6장 참고)
```

3. 디렉토리 구조

```
src/
├── components/
│   ├── game/
│   │   ├── GameWorld.tsx          레이어 조립, 잔디 클릭, 주민 렌더
│   │   ├── WorldObject.tsx       ★ 모든 월드 오브젝트의 공통 껍데기
│   │   ├── WalkingCharacter.tsx   ★ 캐릭터 한 명의 그리기 (시트 칸 + 표정)
│   │   ├── Villager.tsx          agent 상태 -> WalkingCharacter 로 연결
│   │   ├── Background.tsx        배경 이미지 (없으면 하늘/언덕 placeholder)
│   │   ├── StaticEnvironment.tsx  고정 지형지물
│   │   ├── PlaceableObject.tsx   사용자 장식물
│   │   ├── NPC.tsx               NPC (현재 데이터 없음)
│   │   ├── Sprite.tsx            에셋 있으면 img, 없으면 placeholder
│   │   ├── SpriteInspector.tsx    개발용 패널 (DEV 빌드에서만)
│   │   └── placeholders/         종류별 임시 SVG 도형
│   ├── diary/ DiaryEditor · DiaryDetail
│   └── ui/      TopBar · BottomBar · PanelSheet
├── data/
│   ├── characterFrames.ts         순수 데이터 (로직 없음)
│   ├── expressions.ts            표정 에셋 + 감정 매핑
│   ├── worlds.ts                 2026-09 월드 정의
│   ├── emotions.ts · chatLines.ts · seedDiaries.ts · characterParts.ts
│   └── assets/                   배경·장식물·NPC 에셋 레지스트리
├── systems/
│   ├── depthSystem.ts            ★ React 를 import 하지 않는 순수 로직
│   │   │   y -> scale, z-index
│   ├── agentSystem.ts           IDLE / WALK / TALK 상태 기계
│   ├── walkSystem.ts            방향 선택(히스테리시스), 프레임 계산
│   ├── worldSystem.ts           walkable / blocked 영역 판정
│   ├── characterSystem.ts       스폰 위치 결정
│   ├── diarySystem.ts           일기 -> 감정 -> 캐릭터 생성/갱신
│   ├── emotionAnalyzer.ts       텍스트 -> 감정 (AI 로 교체 예정)
│   ├── saveSystem.ts            localStorage
│   └── assetSystem.ts           에셋 존재 확인 + 캐싱
├── store/      gameStore · debugStore
├── hooks/      useVillagers(게임 루프) · useAssetReady
├── types/index.ts
├── styles/     global · game · ui
├── assets/character/
│   │   └── ★ 원본 아트 (서빙 안 됨, 편집용)
│   ├── master/character_master.png
│   ├── idle/{front,back,left,right}.png
│   ├── walk/{front,back,left,right}/NN.png
│   ├── expressions/{neutral,happy,sad,angry}.png
│   ├── reference/               디자인 참고 시트 4장
│   └── clothes/                 (비어 있음)
├── public/assets/
│   │   └── ★ 브라우저가 실제로 받는 파일
│   ├── character/walk-{front,back,left,right}.webp
│   ├── character/expressions/*.webp
│   └── worlds/2026-09/background.webp
└── scripts/pack-sprites.mjs      낱장 -> 시트 + 보정값 생성
```

4. 반드시 지켜야 할 규칙 5가지

#	규칙	이유
1	좌표는 전부 normalized 0~1 . 픽셀을 저장하지 않는다.	화면은 16:9 고정이고 크기가 변해도 배치가 유지돼야 한다.
2	<code>systems/</code> 는 React 를 import 하지 않는다.	감정 분석기를 AI API로 통째로 바꾸거나, 나중에 렌더러를 Canvas로 옮길 때 그대로 재사용하기 위해.
3	에셋 경로를 컴포넌트에 하드코딩 하지 않는다. <code>data/assets</code> 조회 함수를 쓴다.	그림이 바뀌어도 게임 로직을 안 건드리기 위해.
4	<code>src/data/characterFrames.ts</code> 는 생성 파일 이다.	<code>npm run pack-sprites</code> 가 덮어쓴다. 손으로 고치면 다음 실행에 날아간다.
5	<code>src/assets</code> 는 서빙되지 않는다. 게임이 읽는 건 <code>public/assets</code> 다.	Vite는 <code>public/</code> 만 경로 그대로 서빙한다. <code>src/</code> 아래는 import 해야 번들에 들어간다.

좌표계 한 장 요약

화면	16:9 고정. CSS <code>aspect-ratio</code> + 컨테이너 쿼리(<code>cqw</code>) 로 내부가 같이 스케일됨
위치	<code>x</code> , <code>y</code> 는 0~1. <code>left/top</code> 을 % 로 지정
앵커	이미지마다 "바닥에 닿는 지점"(<code>anchorX/anchorY</code>)을 따로 가짐 나무는 밑동, 캐릭터는 발. <code>transform-origin</code> 도 여기에 맞춤
깊이	<code>y</code> 가 클수록(앞쪽) <code>scale</code> 크고 <code>z-index</code> 높음 <code>scale = 0.72 ~ 1.18</code> , <code>zIndex = y * 10000</code> 배경 <code>y=0.25</code> 를 지평선으로 잡음 (쿼터뷰라 위쪽이 원경)
캐릭터 크기	화면 높이의 8% (<code>CHARACTER_HEIGHT_FRACTION</code>)

5. 시스템별 설명

파일	하는 일 · 알아둘 점
depthSystem	y 하나로 scale과 z-index를 계산해 2.5D를 만든다. 실제 3D가 아니다. 모든 오브젝트가 <code>WorldObject</code> 를 통해 이 걸 거친다.
agentSystem	IDLE → WALK → TALK 상태 기계. 각자 자기 자리 반경 0.13 안에서 배회하다가, 두 명이 0.075 이내로 가까워지고 둘 다 한가하면 멈춰 서서 5초간 대화한다. 끝나면 9초 쿨다운. 대사는 번갈아 뜬다.
walkSystem	이동 벡터 → 방향. 대각선에서 방향이 떨어지 않게 히스테리시스 1.35배를 둔다. 프레임은 <code>sequence</code> 배열을 순환한다.
worldSystem	walkable / blocked 사각 영역 판정. 물리 충돌 대신 영역 데이터만 쓴다. <code>furthestWalkablePoint</code> 가 목적지뿐 아니라 경로도 0.008 간격으로 검사해서, 연못이나 건물을 통과하지 않고 갈 수 있는 가장 먼 지점까지만 간다.
diarySystem	일기 저장 → 감정 분석 → 그 날짜 캐릭터 생성 또는 갱신. 같은 날짜면 새로 만들지 않고 감정만 바꾼다. 분석기를 인자로 받으므로 AI 구현으로 교체 가능.
emotionAnalyzer	지금은 한국어 키워드 규칙 기반. <code>analyzeDiary(text) → {emotion, confidence}</code> 시그니처만 지키면 통째로 교체된다. AI 연동 시 여기만 바꾸면 됨.
saveSystem	localStorage 접근은 여기서만. 사파리 프라이빗 모드처럼 막힌 환경에서도 게임이 죽지 않도록 예외를 밖으로 던지지 않는다.
useVillagers	게임 루프는 이 한 곳뿐이다. 캐릭터마다 rAF를 돌리면 수가 늘수록 감당이 안 되므로 전원을 한 루프에서 처리한다. delta time 기반이고, 탭 복귀 시 순간이동을 막는 상한(0.05초)이 있다.

6. 에셋 파이프라인 — 가장 중요

캐릭터 그림을 바꿀 때는 반드시 이 절차를 따르세요. 낱장 원본을 `public/` 에 그냥 넣으면 안 됩니다. 원본 32장은 25MB라 브라우저에 그대로 내려보낼 수 없습니다.

1. 낱장 PNG 를 넣는다 (투명 배경, 발바닥이 이미지 맨 아래)
`src/assets/character/walk/front/01.png ... NN.png`
`src/assets/character/walk/{back,left,right}/` 도 같은 방식
2. `npm run pack-sprites`
3. 스크립트가 자동으로:
 - 방향별 4x2 시트로 묶어 WebP 로 저장 -> `public/assets/character/walk-{dir}.webp`
 - 칸마다 알파 바운딩 박스를 실측
 - 발 기준점(`footX/footY`), 크기 보정(`scale`), 머리 박스(`headX/headY/headW`) 계산
 - 걸음 위상을 보고 재생할 프레임(`sequence`) 선택
 - `src/data/characterFrames.ts` 를 다시 씀

왜 보정값이 필요한가

AI로 만든 그림은 프레임마다 캐릭터의 위치와 크기가 어긋나 있습니다. 실측해 보면 머리 중심이 최대 30px 흘러가고, 발 높이가 23px 흔들리고, 전체 높이가 12% 차이 납니다. 그냥 재생하면 캐릭터가 미끄러지다 튀어오릅니다. 스크립트가 이 편차를 재서 프레임마다 앵커와 스케일로 상쇄합니다.

표정은 어떻게 붙는가

표정은 **머리 전체 그림**이고, 걷기 프레임의 머리 위에 덮어씹웁니다. 표정 이미지 안의 머리 영역 비율(`EXPRESSION_HEAD_BOX` , 실측 $x=7.5\%$ $y=9.4\%$ $w=83.5\%$)과 프레임의 머리 박스를 겹칩니다. 덤으로 프레임마다 달랐던 모자 크기까지 일정해집니다.

정면(front)일 때만 표정이 붙습니다. 받은 표정이 정면 얼굴이라 옆·뒤에 없으면 얼굴이 반대로 붙습니다. 옆·뒤 표정이 생기면 `WalkingCharacter.tsx` 의 `direction === 'front'` 조건을 넓히면 됩니다.

감정 → 표정 매핑

감정 6종	표정 4종	(data/expressions.ts)
NORMAL	-> neutral	
HAPPY	-> happy	
JOYFUL	-> happy	<- 전용 그림이 없어 묶여둔 상태
SAD	-> sad	
ANGRY	-> angry	
ANNOYED	-> angry	<- 전용 그림이 없어 묶여둔 상태

7. 지금 동작하는 것 / 아직 아닌 것

기능	상태	비고
일기 작성	동작	하단 "오늘 기록". 타이핑 중 감정 미리보기, 저장 시 캐릭터 생성
감정 분석	동작 (규칙 기반)	AI 교체 지점은 <code>emotionAnalyzer.ts</code>
캐릭터 생성·갱신	동작	같은 날짜 재작성 시 표정만 변경
저장	동작	localStorage, 새로고침 후 유지
자유 배회	동작	walkable 영역 안에서만
캐릭터 간 대화	동작	마주 보고 말풍선 번갈아
4방향 걷기	동작	좌우 반전 없이 방향별 그림 사용
표정 표시	부분	정면 + 일기 상세 팝업만
클릭 이동	동작	잔디 클릭 시 오늘 캐릭터가 이동. 경로가 막히면 갈 수 있는 데까지만
일기 상세	동작	캐릭터 클릭 시
상점 · 인벤토리	미구현	버튼은 있고 "다음 단계" 안내만 뜸
캐릭터 꾸미기	미구현	<code>clothes/</code> 폴더가 비어 있음
공간 꾸미기	미구현	장식물 시스템은 있으나 배치 UI 없음. 현재 월드 데이터에 장식물 0개
NPC 행동	미구현	NPC 시스템은 있으나 데이터 0개
월 전환 (10월)	미구현	구조는 준비됨 — <code>worlds.ts</code> 와 <code>worldAssets.ts</code> 에 키 하나씩 추가

배경(공원 그림)에 상점·연못·다리·벤치가 이미 그려져 있어, 환경/장식물 오브젝트는 현재 비어 있습니다. 그래서 캐릭터가 벤치 뒤로 들어갈 수 없습니다. 배경은 항상 맨 뒤에 깔리기 때문입니다. 그 자리들은 blocked 영역으로 막아 두었습니다.

8. 알려진 문제와 측정값

걷기 원화가 걷기 사이클이 아니다 (back · left · right)

가장 큰 미해결 문제입니다. 측면 8프레임의 보폭(키 대비)을 재면 36~46%인데, 정상 걷기라면 다리를 모은 자세(10% 안팎)와 벌린 자세(45%)를 오가야 합니다. **다리가 한 번도 모아지지 않습니다.** 걷는 동작이 아니라 같은 자세 8장을 갈아끼우는 것이라, 걷는 게 아니라 떠는 것처럼 보입니다.



구 측면 걷기 8프레임 — 전부 다리를 벌린 거의 같은 자세다

코드로는 **자세 차이가 크고 크기 편차가 작은 2장만 골라** 저속(3fps)으로 재생해 완화했습니다. 머리 크기 출력임이 3.5~6.7%에서 0.5~2.4%로 줄었습니다. 근본 해결은 그림을 다시 뽑는 것입니다.

front는 해결됐습니다. 새로 받은 4장은 발 위상이 $-0.1\% \rightarrow +5.5\% \rightarrow -0.3\% \rightarrow +4.3\%$ 로 "모음 → 내딛음"이 교대합니다. 스크립트가 이를 감지해 4장 전부 재생합니다. **back · left · right도 같은 식으로 4장씩 받으면 됩니다.**

새 걷기 그림 요청 문구 (그대로 쓰면 됩니다)

같은 캐릭터 걷기 4프레임, 정확히 이 순서로:

1. 접지 — 왼발 앞으로 내딛고 오른발 뒤, 다리 벌어짐
2. 지나감 — 두 다리가 거의 모아지고 오른발이 살짝 들림
3. 접지 — 오른발 앞, 왼발 뒤 (1번의 좌우 반대)
4. 지나감 — 두 다리 모아지고 왼발 살짝 들림

조건: 4장 모두 같은 캔버스 크기, 같은 캐릭터 크기, 모자 크기 동일,
배경 투명, 발바닥이 이미지 맨 아래

그 밖

- **캔버스 크기가 방향마다 다릅니다.** front 1086×1448, right 1122×1402, back·left 1024×1536. 코드가 `refHeight` 로 각자 정규화해 흡수하지만, 통일하면 더 깨끗합니다.
- **정지 자세가 없습니다.** 멈추면 걷기 프레임 하나로 섭니다. `idle/` 에 4장이 있으나 아직 연결하지 않았습니다.
- **속도와 fps가 안 맞습니다.** 원래는 보폭에 맞춰야 발이 안 미끄러지는데, 지금 그림에는 맞춤 보폭이 없어 보기 좋은 값으로만 잡아 두었습니다. 제대로 된 걷기가 들어오면 다시 맞춰야 합니다.

9. 합정 — 모르면 다시 뵙습니다

합정	내용
레이어가 클릭을 삼킴	<code>.layer--objects</code> 는 화면 전체를 덮으므로 <code>pointer-events: none</code> 이어야 합니다. 그러지 않으면 잔디 클릭이 전부 막힙니다. 클릭이 필요한 오브젝트만 <code>pointer-events: auto</code> 로 되살립니다. (이 버그로 클릭 이동이 통째로 먹통이었던 적이 있습니다)
저장 직후 상태 초기화	<code>DiaryEditor</code> 의 <code>useEffect</code> 의존성에 <code>diaries</code> 를 넣으면, 저장하는 순간 값이 바뀌어 effect가 돌고 방금 쓴 내용이 되돌아갑니다. 의존성은 <code>date</code> 만이어야 합니다.
신규/수정 구분	저장 후에는 이미 기존 항목이 생기므로, "새로 만들었는지"는 저장 시점에 따로 기억 해야 합니다. 렌더 중에 판단하면 항상 "수정"으로 나옵니다.
생성 파일을 손으로 수정	<code>characterFrames.ts</code> 는 <code>pack-sprites</code> 가 덮어씹습니다. 보정값을 손보고 싶으면 스크립트의 <code>FACE_TUNE</code> 이나 선택 로직을 고치세요.
HMR 중 자동 테스트	파일을 고친 직후 headless 테스트를 돌리면 모듈이 반만 갱신된 상태라 결과가 튼니다. 한 번 더 돌려서 재현되는지 확인하세요. (방향 정확도가 0/28로 나왔다가 재실행에서 172/172이 된 적이 있습니다)
오늘 날짜가 고정	<code>utils/date.ts</code> 의 <code>TODAY</code> 가 <code>'2026-09-21'</code> 로 박혀 있습니다. 9월 월드만 있어서입니다. 월이 늘면 <code>toDateKey(new Date())</code> 로 바꾸세요.
첫 실행 시드	저장된 데이터가 없으면 <code>seedDiaries.ts</code> 의 지난 4일치로 마을을 채웁니다. 빈 마을을 보려면 <code>localStorage</code> 를 지우고 시드를 비우세요.

10. 이미 내려진 결정 (다시 논의하지 마세요)

결정	배경
게임 엔진 안 씀	Unity 사용 금지가 최초 요구사항. 웹 기술만 사용.
서빙 에셋은 WebP	PNG 원본은 <code>src/assets</code> 에 보관, <code>public/</code> 에는 WebP. 5.5MB → 0.8MB.
좌우 반전 안 씀	<code>left·right</code> 각각 별도 그림을 사용. 초기에는 반전으로 만들었으나 실제 그림이 생겨 폐기.
reference/ 유지	주신 구조에는 없지만 디자인 참고 시트를 보관하기 위해 유지.
표정 4종에 감정 6종 묶음	JOYFUL·ANNOYED 전용 그림이 나오면 <code>expressions.ts</code> 에서 풀면 됨.
캐릭터는 모자+노란 스웨터	<code>master/character_master.png</code> 가 기준. 중간에 후드티 버전이 섞였다가 전부 교체함.
단계별 진행	한 번에 다 만들지 않고 단계마다 사용자가 실행 화면을 확인한 뒤 다음으로 넘어감.

11. 다음에 할 일 (우선순위)

- 1. back · left · right 걷기 4프레임씩 받기 — 8장 위 요청 문구 사용. 받으면 폴더에 넣고 `npm run pack-sprites` 만 실행.
- 2. 속도 · fps 재조정 — 제대로 된 걷기가 들어오면 발이 미끄러지지 않도록 맞춤. 개발 패널(우하단 "스프라이트 확인")의 슬라이더로 실시 간 조절 가능.
- 3. idle 4장 연결 — 멈춰 있을 때 걷기 프레임 대신 정지 자세 사용.
- 4. 앞·뒤 표정 — 걸어갈 때도 표정이 유지되게.
- 5. 꾸미기 / 상점 — 하단 버튼 3개가 아직 플레이스홀더. `clothes/` 에셋 필요.
- 6. NPC 행동 — IDLE / LOOK_AROUND / WALK / SIT 상태 기계. `agentSystem` 을 참고하면 거의 그대로 쓸 수 있음.
- 7. 10월 월드 — `worlds.ts` + `worldAssets.ts` 에 키 하나씩.
- 8. AI 감정 분석 — `emotionAnalyzer.ts` 만 교체.

12. 검증 방법

이 프로젝트는 눈으로만 확인하지 않고 **headless Chrome**으로 **실제 조작하며 수치를 재는** 방식으로 검증해 왔습니다. DOM 기반이라 가능한 방법이고, 앞으로도 같은 방식을 권합니다.

무엇을	어떻게
방향 정확도	주민 전원의 위치를 두 번 읽어 이동 벡터를 구하고, 표시 중인 시트 파일명과 대조. 대각선은 히스테리시스 때문에 제외. (최근 160/160)
깊이 정렬	모든 <code>.world-object</code> 의 <code>zIndex</code> 와 <code>transform scale</code> 을 읽어 y에 대해 단조 증가하는지 확인.
프레임 흔들림	날장의 알파 바운딩 박스를 재서 머리 폭·전체 높이 편차를 % 로 계산. 온니언 스킨 이미지로 눈으로도 확인.
걷기 사이클 유무	발 구간의 좌우 최저점 차이(위상)를 프레임마다 측정. 정상 걷기는 10~20%, 가짜는 0~5%.
일기 흐름	실제로 버튼을 누르고 타이핑해서 저장 → 캐릭터 수 증가 → 새로고침 후 유지 → 같은 날짜 재작성 시 수 불변을 확인.

매번 `npm run build` (타입 검사 포함)와 콘솔 에러 0건을 함께 확인했습니다. 이 문서 작성 시점 기준 모두 통과 상태이며, 워킹트리는 깨끗합니다.